
MODULE *vchan*

EXTENDS *Naturals, Sequences, FiniteSets*

CONSTANT *Byte* 0 .. 255, but overridable for modelling purposes. Consider especially $1 \dots \max(\text{Len}(\text{Sent})) + \text{MaxWriteLen}$

The size of the ring buffer in bytes.

CONSTANT *BufferSize*

ASSUME *BufferSize* $\in \text{Nat} \setminus \{0\}$

The most data a sender will try to send in one go.

CONSTANT *MaxWriteLen*

ASSUME *MaxWriteLen* $\in \text{Nat} \setminus \{0\}$

The most data a receiver will try to read in one go.

CONSTANT *MaxReadLen*

ASSUME *MaxReadLen* $\in \text{Nat} \setminus \{0\}$

Endpoint automata

Idle $\triangleq$ "Idle"

Writing $\triangleq$ "Writing"

AfterWriting $\triangleq$ "AfterWriting"

Reading $\triangleq$ "Reading"

AfterReading $\triangleq$ "AfterReading"

Blocked $\triangleq$ "Blocked"

Done $\triangleq$ "Done"

SenderStates $\triangleq \{\text{Idle}, \text{Writing}, \text{AfterWriting}, \text{Blocked}, \text{Done}\}$

ReceiverStates $\triangleq \{\text{Idle}, \text{Reading}, \text{AfterReading}, \text{Blocked}, \text{Done}\}$

$\text{Min}(x, y) \triangleq \text{IF } x < y \text{ THEN } x \text{ ELSE } y$

Take(*m*, *i*) is the first *i* elements of message *m*.

Take(*m*, *i*) $\triangleq \text{SubSeq}(m, 1, i)$

Everything except the first *i* elements of message *m*.

Drop(*m*, *i*) $\triangleq \text{SubSeq}(m, i + 1, \text{Len}(m))$

VARIABLES

history variables (for modelling and properties)

Sent,

Got,

The remaining data that has not yet been added to the buffer:

msg,

status of the endpoints.

<i>SenderLive</i> ,	init true, cleared by sender
<i>ReceiverLive</i> ,	init true, cleared by receiver
<i>SenderState</i> ,	{ <i>Idle</i> , <i>Writing</i> , <i>AfterWriting</i> , <i>Blocked</i> , <i>Done</i> }
<i>ReceiverState</i> ,	{ <i>Idle</i> , <i>Reading</i> , <i>AfterReading</i> , <i>Blocked</i> , <i>Done</i> }

NotifyWrite is a shared flag that is set by the receiver to indicate that it wants to know when data has been written to the buffer. The sender checks it after adding data. If set, the sender clears the flag and notifies the receiver using the event channel. This is represented by *ReceiverIT* being set to TRUE. It becomes FALSE when the receiver handles the event.

<i>NotifyWrite</i> ,	Set by receiver, cleared by sender
<i>ReceiverIT</i> ,	Set by sender, cleared by receiver

Buffer represents the data in transit from the sender to the receiver.
Buffer,

NotifyRead is a shared flag that indicates that the sender wants to know when some data has been read and removed from the buffer (and, therefore, that more space is now available). If the receiver sees that this is set after removing data from the buffer, it clears the flag and signals the sender via the event channel, represented by *SenderIT*.

<i>NotifyRead</i> ,	Set by sender, cleared by receiver
<i>SenderIT</i>	Set by receiver, cleared by sender

The type of the entire message the client application sends, of size up to *MaxWriteLen*.

This is a non-empty bounded *Seq(Byte)*.

$MSG(len) \triangleq \text{UNION } \{[1 \dots N \rightarrow \text{Byte}] : N \in 1 \dots len\}$

Override *MSG* with this to limit *Sent* to the form $\langle 1, 2, 3, \dots \rangle$. This is much faster to check and will still detect any dropped or reordered bytes.

$MSG(len) \triangleq \{[x \in 1 \dots N \mapsto Len(Sent) + x] : N \in 1 \dots len\}$

$MESSAGE \triangleq MSG(MaxWriteLen)$

$vars \triangleq \langle Sent, Got, SenderLive, ReceiverLive, SenderState, ReceiverState, Buffer, msg, NotifyWrite, ReceiverIT \rangle$

Initial state.

$Init \triangleq \wedge SenderLive = \text{TRUE}$
 $\wedge ReceiverLive = \text{TRUE}$
 $\wedge ReceiverState = \text{Idle}$
 $\wedge SenderState = \text{Idle}$
 $\wedge Got = \langle \rangle$
 $\wedge Sent = \langle \rangle$
 $\wedge msg = \langle \rangle$
 $\wedge Buffer = \langle \rangle$
 $\wedge NotifyWrite = \text{FALSE}$
 $\wedge ReceiverIT = \text{FALSE}$
 $\wedge NotifyRead = \text{FALSE}$

$\wedge \text{SenderIT} = \text{FALSE}$

Transition *Idle* $\rightarrow$ *Done*. Receiver is no longer live.

This transition is actually never enabled until *ReceiveClose* is implemented.

$\text{SenderIdle1} \triangleq \wedge \text{SenderLive}$
 $\wedge \text{SenderState} = \text{Idle}$
 $\wedge \neg \text{ReceiverLive}$
 $\wedge \text{SenderState}' = \text{Done}$
 $\wedge \text{UNCHANGED} \langle \text{Sent}, \text{Got}, \text{SenderLive}, \text{ReceiverLive}, \text{ReceiverState}, \text{Buffer}, \text{msg}, \text{NotifyWrite} \rangle$

Transition *Idle* $\rightarrow$ *Writing*. A new message is to be sent.

$\text{SenderIdle2} \triangleq \wedge \text{SenderLive}$
 $\wedge \text{SenderState} = \text{Idle}$
 $\wedge \exists m \in \text{MSG}(\text{MaxWriteLen}) :$
 $\wedge \text{msg}' = m$
 $\wedge \text{Sent}' = \text{Sent} \circ m$
 $\wedge \text{SenderState}' = \text{Writing}$
 $\wedge \text{UNCHANGED} \langle \text{Got}, \text{SenderLive}, \text{ReceiverLive}, \text{ReceiverState}, \text{Buffer}, \text{NotifyWrite}, \text{Receiver} \rangle$

Transition *Writing* $\rightarrow$ *AfterWriting*. Some prefix of *msg* is added to the buffer (without overrunning it)

$\text{SenderWrite1} \triangleq \wedge \text{SenderLive}$
 $\wedge \text{SenderState} = \text{Writing}$
 $\wedge \exists k \in 1 \dots \text{Min}(\text{Len}(\text{msg}), \text{MaxWriteLen}) :$
 $k = \text{nb d'octets dans msg}$
 $\wedge k \leq \text{BufferSize} - \text{Len}(\text{Buffer})$ on n'écrit pas plus que la place dispo
 $\wedge \text{msg}' = \text{Drop}(\text{msg}, k)$
 $\wedge \text{Buffer}' = \text{Buffer} \circ \text{Take}(\text{msg}, k)$
 $\wedge \text{SenderState}' = \text{AfterWriting}$
 $\wedge \text{UNCHANGED} \langle \text{Sent}, \text{Got}, \text{SenderLive}, \text{ReceiverLive}, \text{ReceiverState}, \text{NotifyWrite}, \text{Receiver} \rangle$

Transition *Writing* $\rightarrow$ *Blocked*. The buffer is full.

$\text{SenderWrite2} \triangleq \wedge \text{SenderLive}$
 $\wedge \text{SenderState} = \text{Writing}$
 $\wedge \text{Len}(\text{Buffer}) = \text{BufferSize}$
 $\wedge \text{SenderState}' = \text{Blocked}$
 $\wedge \text{UNCHANGED} \langle \text{Sent}, \text{Got}, \text{SenderLive}, \text{ReceiverLive}, \text{ReceiverState}, \text{Buffer}, \text{msg}, \text{NotifyWrite} \rangle$

Transition *AfterWriting* $\rightarrow$ *Idle*. *msg* is empty, all bytes have been sent. Set *ReceiverIT* if requested.

$\text{SenderWriteNext1} \triangleq \wedge \text{SenderLive}$
 $\wedge \text{SenderState} = \text{AfterWriting}$
 $\wedge \text{Len}(\text{msg}) = 0$
 $\wedge \text{SenderState}' = \text{Idle}$
 $\wedge \text{UNCHANGED} \langle \text{Sent}, \text{Got}, \text{SenderLive}, \text{ReceiverLive}, \text{ReceiverState}, \text{Buffer}, \text{msg}, \text{NotifyWrite} \rangle$

Transition *AfterWriting* $\rightarrow$ *Blocked*. *msg* is not empty, waiting to send more. Set *ReceiverIT* if requested.

$SenderWriteNext2 \triangleq \wedge SenderLive$
 $\wedge SenderState = AfterWriting$
 $\wedge Len(msg) \neq 0$
 $\wedge SenderState' = Blocked$
 $\wedge UNCHANGED \langle Sent, Got, SenderLive, ReceiverLive, ReceiverState, Buffer, msg, NotifyWrite \rangle$

Transition *Blocked* $\rightarrow$ *Writing*.

initial version: no condition (non-deterministic)

final version: IT received while receiver is live.

$SenderUnblock1 \triangleq \wedge SenderLive$
 $\wedge SenderState = Blocked$
 $\wedge SenderState' = Writing$
 $\wedge UNCHANGED \langle Sent, Got, SenderLive, ReceiverLive, ReceiverState, Buffer, msg, NotifyWrite \rangle$

Transition *Blocked* $\rightarrow$ *Done*.

initial version: no condition (non-deterministic)

final version: IT received while receiver is dead.

$SenderUnblock2 \triangleq UNCHANGED vars$

Transition any state $\rightarrow$ *Done*. Sender is no longer live.

$SenderEnd \triangleq UNCHANGED vars$

Transition *Idle* $\rightarrow$ *Blocked*. Buffer is empty and sender is live.

$ReceiverIdle1 \triangleq \wedge ReceiverLive$
 $\wedge ReceiverState = Idle$
 $\wedge SenderLive$
 $\wedge Len(Buffer) = 0$
 $\wedge ReceiverState' = Blocked$
 $\wedge NotifyWrite' = TRUE$
 $\wedge UNCHANGED \langle Sent, Got, SenderLive, ReceiverLive, SenderState, Buffer, msg, ReceiverIT \rangle$

Transition *Idle* $\rightarrow$ *Reading*. Buffer is not empty.

$ReceiverIdle2 \triangleq \wedge ReceiverLive$
 $\wedge ReceiverState = Idle$
 $\wedge Len(Buffer) \neq 0$
 $\wedge ReceiverState' = Reading$
 $\wedge UNCHANGED \langle Sent, Got, SenderLive, ReceiverLive, SenderState, Buffer, msg, NotifyWrite \rangle$

Transition *Idle* $\rightarrow$ *Done*. Sender is dead and buffer is empty.

$ReceiverIdle3 \triangleq UNCHANGED vars$

Transition *Reading* $\rightarrow$ *AfterReading*. Extract some bytes from buffer.

$ReceiverRead \triangleq \wedge ReceiverLive$
 $\wedge ReceiverState = Reading$
 $\wedge \exists k \in 1 \dots Min(Len(Buffer), MaxReadLen) :$

$\wedge Got' = Got \circ Take(Buffer, k)$ copier le début du buffer
 $\wedge Buffer' = Drop(Buffer, k)$ vider le buffer des k premiers
 $\wedge ReceiverState' = AfterReading$
 $\wedge \text{UNCHANGED } \langle Sent, SenderLive, ReceiverLive, SenderState, msg, NotifyWrite, ReceiverIT \rangle$

Transition *AfterReading* $\rightarrow$ *Idle*. Back to *Idle*. Set *SenderIT* if requested.

$ReceiverReadNext \triangleq \wedge ReceiverLive$
 $\wedge ReceiverState = AfterReading$
 $\wedge ReceiverState' = Idle$
 $\wedge \text{UNCHANGED } \langle Sent, Got, SenderLive, ReceiverLive, SenderState, Buffer, msg, NotifyWrite, ReceiverIT \rangle$

Transition *Blocked* $\rightarrow$ *Idle*.

initial version: no condition (non-deterministic)

final version: IT received.

$ReceiverUnblock \triangleq \wedge ReceiverLive$
 $\wedge ReceiverState = Blocked$
 $\wedge ReceiverState' = Idle$
 $\wedge \text{UNCHANGED } \langle Sent, Got, SenderLive, ReceiverLive, SenderState, Buffer, msg, NotifyWrite, ReceiverIT \rangle$

Transition any state $\rightarrow$ *Done*. Receiver is no longer live.

$ReceiverEnd \triangleq \text{UNCHANGED } vars$

Asynchronous abort of any endpoint.

Sender abruptly becomes dead. It sends an IT to receiver on dying.

$SenderClose \triangleq \text{UNCHANGED } vars$

Receiver abruptly becomes dead. It sends an IT to sender on dying.

$ReceiverClose \triangleq \text{UNCHANGED } vars$

Spurious interruption

A *SenderIT* spuriously appears.

A *ReceiverIT* spuriously appears.

$SenderNext \triangleq SenderIdle1 \vee SenderIdle2 \vee SenderWrite1 \vee SenderWrite2 \vee SenderWriteNext1 \vee SenderWriteNext2$

$ReceiverNext \triangleq ReceiverIdle1 \vee ReceiverIdle2 \vee ReceiverIdle3 \vee ReceiverRead \vee ReceiverReadNext \vee ReceiverWrite$

$Next \triangleq \vee SenderNext \vee ReceiverNext$
 $\vee SenderClose \vee ReceiverClose$

Weak fairness on sender and weak fairness on receiver: both will progress as long as they do not deadlock.

No fairness on $\{Sender, Receiver\}Close$ or spurious IT: these events may never occur.

$$Fairness \triangleq WF_{vars}(SenderNext) \wedge WF_{vars}(ReceiverNext)$$

$$Spec \triangleq Init \wedge \Box[Next]_{vars} \wedge Fairness$$

$$TypeOk \triangleq$$

$$\wedge Sent \in Seq(Byte)$$

$$\wedge Got \in Seq(Byte)$$

$$\wedge Buffer \in Seq(Byte)$$

$$\wedge SenderLive \in BOOLEAN$$

$$\wedge ReceiverLive \in BOOLEAN$$

$$\wedge NotifyWrite \in BOOLEAN$$

$$\wedge SenderIT \in BOOLEAN$$

$$\wedge NotifyRead \in BOOLEAN$$

$$\wedge ReceiverIT \in BOOLEAN$$

$$\wedge SenderState \in \{Idle, Writing, AfterWriting, Blocked, Done\}$$

$$\wedge ReceiverState \in \{Idle, Reading, AfterReading, Blocked, Done\}$$

$$\wedge msg \in Seq(Byte)$$

Whatever we receive is the same as what was sent (*i.e.* *Got* is a prefix of *Sent*)

$$Integrity \triangleq (Take(Sent, Len(Got)) = Got)$$

Any data that the write function reports has been sent successfully (*i.e.* data in *Sent* when it is back at “ready”) will eventually be received (if the receiver doesn’t close the connection). In particular, this says that it’s *OK* for the sender to close its end immediately after sending some data.

$$Availability \triangleq$$

$$\forall x \in 0 \dots Cardinality(Byte) : x = Len(Sent) \wedge SenderState = Idle \leadsto (Len(Got) \geq x)$$

If either side closes the connection, both end up in their *Done* state

$$ShutdownOK \triangleq (\neg SenderLive \vee \neg ReceiverLive) \leadsto (SenderState = Done \wedge ReceiverState = Done)$$

If both ends never close the connection (and *Sent* is finite), then the receiver eventually gets all the sent bytes.

$$NoLoss \triangleq \Diamond\Box(Got = Sent)$$
